EaglesEye — Stack Study Guide

The four technologies you don't know yet, taught against your own running dev stack

For: Moataz · Skipped: Kafka, Keycloak (you know them) · Prereq: `cd infra\dev && docker compose up -d` · Version 1.0 — 07 Aug 2026

How to use this guide: each chapter has four parts — *what it is*, *the 20% you actually need*, *hands-on exercises that run against your real EaglesEye containers*, and *ordered study links* for going deeper. Do the hands-on parts in order; each takes 20–40 minutes. Everything here is traced to the sprint task where you'll use it.

Chapter	Technology	Role in EaglesEye	Needed by
1	TimescaleDB	positions storage: hypertables, compression, retention	Sprint 2 (T-206, T-207)
2	PostGIS	geofences: geometry, spatial indexes, point-in-polygon	Sprint 6 (T-605, T-606)
3	Valkey	live vehicle state, alert cooldowns, pub/sub fan-out	Sprint 5 (T-501), Sprint 7 (T-703)
4	MQTT / Mosquitto	generic JSON device path + your phone (OwnTracks)	Now (T-110), Sprint 3 (T-303)
5	Later stack	OSRM, MapLibre, OwnTracks — links only for now	Sprints 5–6

---

1. TimescaleDB — time-series on PostgreSQL

Where every GPS position lives · Sprint 2 · tasks T-206, T-207

What it is

TimescaleDB is a PostgreSQL extension — not a separate database. You keep normal SQL, joins, indexes and tools, and it adds one core concept: the **hypertable**. A hypertable looks like an ordinary table but is automatically partitioned behind the scenes into **chunks** by time (e.g. one chunk per week). Inserts land in the small "hot" chunk, old chunks get compressed ~10×, and dropping old data is instant (drop the chunk, no DELETE scan).

Why EaglesEye uses it: at 500 vehicles × 1 position/10 s you write ~4M rows/day. A plain table slows down and can't drop 12-month-old data cheaply (NFR-06). A hypertable keeps inserts fast forever, makes "last 30 days for vehicle X" a near-sequential read (NFR-07: <3 s), and gives retention for free.

The 20% you need

- `create_hypertable('positions','device_time')` — turns a table into a hypertable
- Chunks — you almost never touch them directly; know they exist and that queries which filter on time only scan relevant chunks
- Compression policy — `segmentby = device_id` is the key decision: rows of the same device compress together, making per-vehicle history reads fast
- Retention — `add_retention_policy` drops chunks older than N months automatically
- `time_bucket('1 hour', device_time)` — GROUP BY for time; powers daily distance/utilisation reports
- Continuous aggregates — materialized views that update incrementally; you'll use them in Sprint 8 for report speed

Hands-on — against your ee-timescaledb container

```
docker exec -it ee-timescaledb psql -U eagleseye -d eagleseye
```

```
-- 1. a mini positions hypertable
CREATE TABLE demo_positions (
  device_id   TEXT NOT NULL,
  device_time TIMESTAMPTZ NOT NULL,
  lat DOUBLE PRECISION, lon DOUBLE PRECISION,
  speed_kmh REAL
);
SELECT create_hypertable('demo_positions', 'device_time');

-- 2. insert 3 days of fake data for 2 devices (~5,000 rows)
INSERT INTO demo_positions
SELECT 'IMEI-' || d, now() - (i || ' minutes')::interval,
       30.0 + random()*0.2, 31.2 + random()*0.2, random()*90
FROM generate_series(1,2) d, generate_series(1,2500) i;

-- 3. the report query shape you'll write constantly
SELECT device_id, time_bucket('1 hour', device_time) AS hour,
       avg(speed_kmh)::int AS avg_speed, count(*) AS points
FROM demo_positions
WHERE device_time > now() - interval '1 day'
GROUP BY device_id, hour ORDER BY hour DESC LIMIT 10;

-- 4. see the chunks Timescale created for you
SELECT show_chunks('demo_positions');

-- 5. compression: THE production setting for EaglesEye
ALTER TABLE demo_positions SET (
  timescaledb.compress,
  timescaledb.compress_segmentby = 'device_id',
  timescaledb.compress_orderby   = 'device_time DESC');
SELECT add_compression_policy('demo_positions', INTERVAL '7 days');

-- 6. retention = NFR-06 (12 months) in one line
SELECT add_retention_policy('demo_positions', INTERVAL '12 months');

-- cleanup when done
DROP TABLE demo_positions;
```

Study links, in order

1. Timescale — Getting Started — official, ~1 hour, do it once
2. Hypertables explained — the mental model

3. Compression — read "About compression"; segmentby/orderby is exactly our T-207 decision
4. Continuous aggregates — skim now, needed in Sprint 8 (reports)
5. Tutorials — the "transport/IoT" ones mirror our exact workload

2. PostGIS — geospatial SQL

How geofences work · Sprint 6 · tasks T-605, T-606 · also map queries everywhere

What it is

PostGIS is the geospatial extension for PostgreSQL — the industry standard for 20+ years. It adds a `geometry` column type (points, polygons, lines), spatial functions (`ST_Contains`, `ST_Distance`, `ST_Buffer`…), and spatial indexes (GiST) that make "which geofence contains this point" fast even with thousands of zones. Traccar does its geofencing in application code; we do it in the database — stronger and simpler.

> **Why EaglesEye uses it:** geofences (FR-GEO) are polygons/circles; every incoming position may need "is this point inside any assigned zone?" answered. PostGIS + GiST index answers that in microseconds, and the same machinery later powers "vehicles near customer site" API queries.

The 20% you need

- **SRID 4326** = plain GPS lat/lon (WGS-84). Every geometry we store uses it. Beware: PostGIS point order is `(lon, lat)` — longitude FIRST. This is the #1 beginner bug.
- `geometry` vs `geography`: geometry = fast planar math (fine for zone containment); geography = accurate metres over the earth (use for distances). Rule of thumb: containment → geometry; distance in metres → cast `::geography`
- `ST_Contains(zone, point)` / `ST_Within` — the geofence check
- `ST_DWithin(a, b, metres)` — "within X metres", uses the index (never write `ST_Distance < x`)
- `ST_GeomFromText('POINT(31.23 30.04)', 4326)` and `ST_AsGeoJSON()` — in and out
- GiST index: `CREATE INDEX ON geofences USING GIST (geom);` — without it everything is a full scan

Hands-on — a real Cairo geofence

```
docker exec -it ee-timescaledb psql -U eagleseye -d eagleseye
```

```
-- 1. a geofence table like Sprint 6 will have
CREATE TABLE demo_geofences (
  id SERIAL PRIMARY KEY, name TEXT,
  geom geometry(Polygon, 4326)
);
CREATE INDEX ON demo_geofences USING GIST (geom);

-- 2. a polygon roughly around Downtown Cairo (lon lat pairs!)
INSERT INTO demo_geofences (name, geom) VALUES ('Downtown Cairo',
  ST_GeomFromText('POLYGON((31.22 30.03, 31.26 30.03, 31.26 30.06,
                            31.22 30.06, 31.22 30.03))', 4326));

-- 3. the geofence evaluator check (T-606): is a vehicle inside?
SELECT name,
  ST_Contains(geom, ST_SetSRID(ST_MakePoint(31.24, 30.045), 4326)) AS tahrir_inside,
  ST_Contains(geom, ST_SetSRID(ST_MakePoint(31.40, 30.10 ), 4326)) AS airport_inside
FROM demo_geofences;

-- 4. distance in metres: geometry vs geography (see the difference!)
SELECT
  ST_Distance(ST_SetSRID(ST_MakePoint(31.24,30.045),4326),
              ST_SetSRID(ST_MakePoint(31.40,30.10),4326))             AS degrees_meaningless,
  ST_Distance(ST_SetSRID(ST_MakePoint(31.24,30.045),4326)::geography,
              ST_SetSRID(ST_MakePoint(31.40,30.10),4326)::geography)::int AS metres_real;

-- 5. "within 500 m of the depot" — the index-friendly way
SELECT name FROM demo_geofences
WHERE ST_DWithin(geom::geography,
      ST_SetSRID(ST_MakePoint(31.245, 30.05),4326)::geography, 500);

DROP TABLE demo_geofences;
```

Study links, in order

1. Introduction to PostGIS workshop — THE resource; free, excellent, do sections 1–17 (skip loading/projection deep-dives on first pass)
2. PostGIS reference — bookmark; you'll grep it for ST_ functions
3. Geometry vs geography chapter — reread before Sprint 6

3. Valkey — in-memory data store (the Redis fork)

Live vehicle state, alert cooldowns, WebSocket fan-out · Sprints 5 & 7 · tasks T-501, T-503, T-703

What it is

Valkey is the Linux Foundation's open-source fork of Redis (BSD licence — kept for our CN-02 rule), fully Redis-compatible: same commands, same clients, same docs. It's an in-memory key-value store with rich data types (strings, hashes, sets, sorted sets) plus pub/sub and per-key expiry (TTL). Think of it as a shared, blazing-fast scratchpad between services — **never** the source of truth. In EaglesEye, losing Valkey means the live map is stale for a few seconds until the enricher repopulates it; nothing is lost.

> **Why EaglesEye uses it:** the live map (FR-TRK) must read "current state of 500 vehicles" hundreds of times per second — that never touches PostgreSQL; it reads Valkey hashes. Alert cooldowns (FR-ALT-06, the anti-noise rule) are one atomic command. And pub/sub fans positions out to WebSocket sessions (T-503).

The 20% you need

- **Hashes** — one hash per vehicle: `HSET vehicle:123 lat 30.04 lon 31.23 status moving`, read with `HGETALL`
- **TTL** — `EXPIRE key seconds` / `SET key val EX 300`; keys vanish automatically
- **The cooldown pattern** (memorise this one): `SET cooldown:speeding:veh123 1 NX EX 600` — NX = only if not exists. Returns OK → first alert, send it. Returns nil → suppressed duplicate. Atomic, no race conditions.
- **Pub/sub** — `SUBSCRIBE positions` / `PUBLISH positions "…"`; fire-and-forget (no replay — that's Kafka's job; know the difference)
- **Sorted sets** — `ZADD last_seen <timestamp> <device>` then `ZRANGEBYSCORE`: "devices silent > 10 min" for the health module
- Single-threaded mental model: every command is atomic; don't store big blobs; everything is O(small)

Hands-on — against your ee-valkey container

```
docker exec -it ee-valkey valkey-cli
```

```
# 1. live vehicle state (what T-501 enricher will write)
HSET vehicle:TRK-102 lat 30.044 lon 31.235 speed 62 status moving ignition 1
HGETALL vehicle:TRK-102
HSET vehicle:TRK-102 speed 0 status idle     # partial update
HGET vehicle:TRK-102 status

# 2. offline detection with TTL
SET heartbeat:TRK-102 1 EX 10       # device pings every 10 s
TTL heartbeat:TRK-102               # watch it count down
EXISTS heartbeat:TRK-102            # 0 after expiry = device offline

# 3. THE cooldown pattern (T-703, FR-ALT-06)
SET cooldown:speeding:TRK-102 1 NX EX 600    # -> OK    (send alert)
SET cooldown:speeding:TRK-102 1 NX EX 600    # -> (nil) (suppress!)

# 4. pub/sub — open a SECOND terminal:
#    docker exec -it ee-valkey valkey-cli
#    SUBSCRIBE positions
# then in the first one:
PUBLISH positions "TRK-102,30.044,31.235,62"

# 5. staleness ranking (health module, T-801)
ZADD last_seen 1754500000 dev-a 1754590000 dev-b
ZRANGEBYSCORE last_seen -inf 1754550000      # devices not seen since cutoff
```

Study links, in order

1. Valkey — data types intro — core reading, ~40 min
2. Valkey command reference — bookmark; try commands in valkey-cli as you read
3. Pub/Sub explained — short; note the "no delivery guarantee" part (that's why Kafka carries the truth)
4. Redis developer docs — everything applies to Valkey; richer tutorials and patterns

4. MQTT + Mosquitto — the lightweight IoT protocol

Generic JSON device path + your phone · NOW (T-110), Sprint 3 (T-303) · FR-ING-04

What it is

MQTT is a tiny publish/subscribe protocol built for devices on bad networks — exactly the world GPS trackers live in. Devices **publish** messages to hierarchical **topics** (like `owntracks/moataz/phone1`); services **subscribe** to topic patterns. A **broker** (Mosquitto for us) sits in the middle and routes. Unlike your Teltonika/GT06 binary TCP protocols, modern and custom devices increasingly just publish JSON over MQTT — that's our fourth ingestion path, and it's how your phone becomes device #1 today.

> **Why EaglesEye uses it:** FR-ING-04 requires a generic JSON/MQTT path so any modern or custom device can join without us writing a binary decoder. It's also the zero-budget door: OwnTracks on your phone speaks MQTT natively (T-110).

The 20% you need

- **Topics** are just names with `/` hierarchy; nobody pre-creates them. Wildcards when subscribing: `+` = one level (`owntracks/+/phone1`), `#` = everything below (`owntracks/#`)
- **QoS** 0 = fire and forget · 1 = at least once (duplicates possible — our dedup handles that) · 2 = exactly once (slow, skip). EaglesEye ingests at QoS 1.
- **Retained message** — broker keeps the last message per topic; new subscribers get it immediately. OwnTracks uses this for "last known location".
- **Last Will & Testament (LWT)** — broker publishes a message on behalf of a client that dies unexpectedly → free device-offline detection (feeds FR-HLT later)
- **Keep-alive** — client pings; broker declares death after 1.5× interval; that's what triggers LWT
- Mosquitto tools: `mosquitto_sub` / `mosquitto_pub` — you'll use them constantly for debugging

Hands-on — against your ee-mosquitto container

```
# Terminal 1 — the spy: watch EVERYTHING crossing the broker
docker exec -it ee-mosquitto mosquitto_sub -t '#' -v

# Terminal 2 — publish like a device would
docker exec -it ee-mosquitto mosquitto_pub -t devices/veh1/pos -m '{"lat":30.04,"lon":31.23,"spd":

# wildcards: subscribe to one level vs whole subtree (terminal 1, Ctrl+C first)
docker exec -it ee-mosquitto mosquitto_sub -t 'devices/+/pos' -v      # any vehicle, pos only
docker exec -it ee-mosquitto mosquitto_sub -t 'devices/#' -v          # everything under devices

# retained message: publish with -r, then subscribe AFTER — you still get it
docker exec -it ee-mosquitto mosquitto_pub -t devices/veh1/last -m '{"lat":30.05}' -r
docker exec -it ee-mosquitto mosquitto_sub -t devices/veh1/last -C 1

# QoS 1 publish
docker exec -it ee-mosquitto mosquitto_pub -q 1 -t devices/veh1/pos -m 'qos1-msg'

# and once OwnTracks is set up on your phone (T-110):
docker exec -it ee-mosquitto mosquitto_sub -t 'owntracks/#' -v
```

Study links, in order

1. HiveMQ — MQTT Essentials — THE course; 10 short parts; parts 1–9 cover everything above
2. Mosquitto documentation — broker config reference (auth & TLS matter in Sprint 3's hardening)
3. OwnTracks Booklet — the phone app's manual; the "MQTT mode" chapter is your T-110 reference, and its JSON payload format is documented here

5. Coming later — bookmark, don't study yet

These arrive in Sprints 5–6. Save the links; reading them now would be premature.

Technology	Role	When	Start here
MapLibre GL JS	the live map in the console	Sprint 5 (T-502)	maplibre.org docs + examples
OSRM	snap GPS paths to roads; the 2% odometer target	Sprint 6 (T-602)	project-osrm.org — see the /match service
Photon	reverse geocoding (coords → address)	Sprint 5 (T-505)	github.com/komoot/photon
Quarkus deep-dives	Kafka + Hibernate + scheduler patterns	ongoing	quarkus.io/guides — read a guide when its task arrives

Suggested schedule

When	Study	Hours	Because
This week	Ch. 4 — MQTT hands-on + HiveMQ parts 1–5	2–3	T-110 (phone) is today's task
Before Sprint 2 (Aug 23)	Ch. 1 — TimescaleDB hands-on + Getting Started	3–4	T-206/T-207 build the positions hypertable
Before Sprint 5 (Oct 04)	Ch. 3 — Valkey hands-on + data types	2	T-501 live state, T-503 fan-out
Before Sprint 6 (Oct 18)	Ch. 2 — PostGIS workshop	4–6	T-605/T-606 geofencing

Honest note: links were curated as of Aug 2026 from official sources; if one moves, the project name + topic in a search engine finds the successor. The hands-on snippets in this guide run against your own containers, so they don't rot.